

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Assignment 2

Due 5:00 p.m., February 6, 2026

IMPORTANT! This is an individual assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the [Academic Honesty in Computing](#) section.

Problem Definition

Expanding on your implementation from Homework 3, develop a simple full-duplex, asynchronous talk utility using named pipes (FIFOs). You will be writing two separate programs: `server.c` and `client.c`.

Requirements

1. Because system calls like `read()` and `fgets()` are blocking, a single-threaded process would freeze while waiting for user input or an incoming network packet. To solve this, each of your programs must utilize `fork()`:
 - Parent process monitors `stdin` and writes to the outgoing FIFO.
 - Child process blocks on the incoming FIFO and immediately renders received data to `stdout`.
2. You must establish two independent FIFOs in a local directory. Because `open()` on a FIFO is a blocking operation, the sequence in which you open these files is critical to avoid a deadlock. So, the server must create the pipes, and the client open the existing pipes.
3. Your implementation must be asynchronous. A user should be able to receive multiple messages while in the middle of typing a response. Use `fflush(stdout)` to ensure that messages appear immediately.
4. When the parent process receives a message containing only the special character ('.'), it writes that message to the pipe and exits. When the child process receives that same character from the pipe, it knows the conversation is over and exits itself.

Programming Notes

- Compile with:

```
$ gcc -Wall -o server server.c
$ gcc -Wall -o client client.c
```

- Open two separate terminals and run:

```
$ ./server      @terminal 1  
$ ./client     @terminal 2
```

Handin

When completed, submit your C source, `server.c` and `client.c`, on the course website.

- **Welcome: Sean**

- [LogOut](#)

